

Commodity-Hardware Execution Path for Bit-Uniform Relational Storage

AVX-512 / NEON / SVE Only — No FPGA, No PIM, No TCAM

Anonymous

July 29, 2026

Abstract

The bit-uniform relational invariant — every cell stored as a single 64-bit word regardless of logical type — is sometimes dismissed as a scheme that only pays off on exotic accelerators such as FPGAs, processing-in-memory (PIM) fabrics, or ternary content-addressable memory (TCAM). This document argues the opposite: the invariant unlocks substantial speedups on stock commodity CPUs, specifically x86-64 cores with AVX-512 and ARM cores with NEON and SVE. We catalogue the relevant SIMD primitives — `_mm512_popcnt_epi64`, `_mm512_ternarylogic_epi64`, `_mm512_cmpeq_epi64_mask`, `_mm_cmp_epi8_mask`, and the ARM SVE predicate-driven compares — and show how each maps onto a bit-uniform kernel. We then present vectorized scan, bit-sliced index, SwissTable-style hash join, and trace-JIT specialization kernels written in Rust against `std::arch` intrinsics. Projected against DuckDB and ClickHouse on five representative workloads, the bit-uniform path delivers 1.5–2× on TPC-H Q1, 5–10× on TPC-H Q6, and 50–100× on similarity joins that no current baseline supports. We close by honestly enumerating what is lost without special hardware and how software mitigations recover most of the gap.

1 Introduction

A common reaction to the bit-uniform relational model is scepticism: “this only matters if you have custom silicon.” The premise of this document is the opposite. We deliberately constrain the design to *commodity CPUs only* — x86-64 cores shipping since Ice Lake (2019) and ARM cores with NEON and, increasingly, SVE. No FPGAs, no processing-in-memory (PIM) fabrics such as UPMEM, no TCAM, no ASICs. Anything that cannot be bought in a mainstream server SKU in 2024 is out of scope. GPUs are mentioned only as future work, not as part of the main design.

The constraint is not as limiting as it first appears. Modern commodity cores ship with extraordinarily wide SIMD units. Ice Lake and later Intel parts, as well as Zen 4 AMD parts, expose 512-bit AVX-512 ports with three independently issueable SIMD execution ports per cycle. The `VPOPCNTDQ` extension delivers eight 64-bit population counts per instruction; `VPTERNLOGQ` fuses any three-input Boolean function into a single instruction; `VPCMQEQQ` and `VPCMPGTQ` produce 8-bit masks of lane-wise equality and greater-than comparisons. ARM NEON offers the scalar-free `cnt` instruction for byte-wise popcount, and SVE adds predicate-driven vectorization with variable lane widths. These are exactly the primitives a bit-uniform engine needs.

The thesis of this document is the following: *even on commodity hardware, bit-uniform storage beats type-specialized storage by 2–5× on analytic workloads*, and by one to two orders of magnitude on a class of approximate-match queries that type-specialized engines do not support at all. The reason is twofold. First, monomorphic 64-bit columns stream through cache lines at peak bandwidth and are trivially vectorizable because every lane holds the same logical shape. Second, the bit-uniform representation turns a large class of expensive operations — Hamming-distance filters, range queries, similarity joins — into bit-parallel Boolean kernels that map directly onto the SIMD primitives above. We quantify this claim in section 8 against DuckDB (Raasveldt and Mühleisen, 2019) and ClickHouse (Schulze et al., 2024).

2 Commodity CPU SIMD Inventory

Before building kernels we inventory the exact instructions available on the target platforms. The bit-uniform design rests on five primitive families: per-lane population count, fused ternary Boolean logic, lane-wise integer compare with mask output, byte-wise compare for hash-table probing, and byte/word move-mask extraction. Each platform exposes these with different names and throughputs, but the abstract shape is portable.

2.1 x86-64 AVX-512 (Ice Lake, Zen 4, and later)

Intel’s Ice Lake client and server parts (2019+) and AMD’s Zen 4 (2022+) implement the full AVX-512 foundation plus the VPOPCNTDQ, VBMI2, and BW extensions. The key intrinsics for our purposes are `_mm512_popcnt_epi64` (eight 64-bit popcounts per instruction, throughput 1 inst/cycle on Ice Lake, $\frac{1}{2}$ inst/cycle on Zen 4 due to its folded 256-bit datapath for some paths), `_mm512_ternarylogic_epi64` (arbitrary three-input Boolean function, throughput 1 inst/cycle), `_mm512_cmpeq_epi64_mask` and `_mm512_cmpgt_epi64_mask` (lane compares with 8-bit mask output, throughput 1 inst/cycle), and `_mm512_movepi8_mask` (extract one bit per byte into a 64-bit register). The VBMI2 extension adds `_mm512_mask_compressstoreu_epi64`, which lets us gather matching lanes into a contiguous output buffer without a scatter, an essential primitive for filtered scans.

2.2 ARM NEON (Cortex-A78, Apple M2, and later)

ARM NEON operates on 128-bit vectors. The `cnt` instruction computes byte-wise popcount across the 16 bytes of a vector in a single cycle; for 64-bit lane popcount we sum the per-byte counts with a widening add and a horizontal reduction, costing roughly 4 instructions per 128-bit vector (two 64-bit lanes). Lane compares (`vceqq_u64`, `vcgtq_u64`) produce full masks rather than compact 8-bit results, which is less convenient than AVX-512 but still usable. Apple’s M2 additionally exposes the undocumented-but-stable AMX unit for matrix work, but we do not rely on it: NEON alone is sufficient for portability across the ARM server ecosystem (Ampere Altra, Graviton 3/4).

2.3 ARM SVE (Graviton 3, Fujitsu A64FX, and later)

SVE decouples vector width from the instruction encoding: the same binary runs on 128-bit, 256-bit, and 512-bit hardware. Its distinguishing feature for bit-uniform work is *predication*: every vector instruction accepts a predicate register that gates per-lane execution. This collapses the compare-then-branch pattern into a single predicated compare followed by a compress-store (**compact**), which is exactly the filtered-scan primitive we need. SVE2 adds *MATCH* and *NMATCH* instructions for string pattern matching, useful for the string-similarity kernels of section 8.

2.4 Throughput of key primitives

Table 1 summarizes the measured or vendor-specified throughput of the primitives our kernels depend on. “Lanes/inst” is the number of 64-bit cells processed per instruction; “inst/cycle” is the sustained issue rate on a single core with both ports available.

The headline number is straightforward: on Ice Lake, a single `_mm512_popcnt_epi64` retires every cycle, processing eight 64-bit cells, for a peak of $8 \times 3.5 \text{ GHz} \approx 28 \text{ Gcells/s/core}$ of pure popcount throughput. Whether the scan kernel can actually sustain that rate depends on memory bandwidth, which we analyze in section 3.

3 Bit-Parallel Scan Kernel

The scan kernel is the load-bearing primitive of the entire system: every filter, every aggregation, every join is, at the bottom, a vectorized walk over a column of 64-bit words. Getting it right is therefore the first and most important engineering task.

Table 1: Throughput of key SIMD primitives on three commodity microarchitectures. Lanes are 64-bit cells; inst/cycle is sustained single-core issue.

Primitive	Lanes/inst	Ice Lake	Zen 4	Apple M2 (NEON)
<code>_mm512_popcnt_epi64</code> (VPOPCNTDQ)	8	1.0	0.5	—
<code>_mm512_ternarylogic_epi64</code> (VPTERNLOGQ)	8	1.0	1.0	—
<code>_mm512_cmpeq_epi64_mask</code> (VPCMPEQQ)	8	1.0	1.0	—
<code>_mm512_cmpgt_epi64_mask</code> (VPCMPGTQ)	8	1.0	1.0	—
<code>_mm512_movepi8_mask</code> (VPMOVMSKB)	64	1.0	1.0	—
NEON cnt (byte popcount)	16	—	—	1.0
NEON vceqq_u64	2	—	—	1.0
SVE predicated compare + compact	4–8	—	—	—

3.1 The naive kernel

The simplest bit-parallel similarity scan computes, for each cell w and a query word q , the Hamming distance $\text{popcount}(w \oplus q)$ and emits the cell if the distance is at or below a threshold τ . In scalar form this is three operations per cell (XOR, popcount, compare), which on a modern x86 core compiling to `popcnt` retires at roughly one cell per cycle, or ~ 3.5 Gcells/s/core. This is the baseline against which we measure the vectorized kernel.

3.2 The vectorized kernel

The vectorized version replaces the per-cell loop with a per-eight-cell loop. One `_mm512_loadu_si512` loads eight cells, one `_mm512_xor_si512` XORs them with the broadcast query, one `_mm512_popcnt_epi64` produces eight per-lane distances, and one `_mm512_cmpgt_epi64_mask` produces an 8-bit match mask. VBMI2’s `compressstore` then writes the surviving cells contiguously. The kernel thus retires four or five instructions per eight cells, and because each of those instructions issues at one per cycle, the compute-bound peak is ~ 8 cells/cycle, or ~ 28 Gcells/s/core at 3.5 GHz.

3.3 Rust implementation

Listing 1 shows the kernel in Rust against `std::arch::x86_64`. The #

target_feature

attribute guards the intrinsics; the caller is expected to feature-detect at startup and dispatch through a function pointer.

```

1 use std::arch::x86_64::*;
2
3 /// Bit-parallel similarity scan: counts cells whose Hamming distance to
4 /// `query` is <= `threshold`, processing 8 u64s per iteration.
5 ///
6 /// Safety: requires AVX-512F, AVX-512VPOPCNTDQ, AVX-512BW. The caller
7 /// must guarantee `data.len() % 8 == 0` and `out_counts.len() == data.len()/8`.
8 #[target_feature(enable = "avx512f,avx512vpopcntdq,avx512bw")]
9 pub unsafe fn scan_popcnt_avx512(
10     data: &[u64],           // bit-uniform column slice, length % 8 == 0
11     query: u64,             // the query word
12     threshold: u64,         // max Hamming distance
13     out_counts: &mut [u8],
14 ) {
15     // Broadcast the query word into every 64-bit lane.
16     let q = _mm512_set1_epi64(query as i64);
17     let thr = _mm512_set1_epi64(threshold as i64);
18     let mut i = 0;
19     while i < data.len() {
20         // 1. Load 8 cells (512 bits) with one unaligned load.

```

```

21     let v = _mm512_loadu_si512(data.as_ptr().add(i) as *const i64);
22     // 2. XOR with query: differing bits are now 1.
23     let diff = _mm512_xor_si512(v, q);
24     // 3. Per-lane population count == Hamming distance per cell.
25     let dist = _mm512_popcnt_epi64(diff);
26     // 4. Produce 8-bit mask of lanes where dist > threshold.
27     let gt_mask = _mm512_cmpgt_epi64_mask(dist, thr);
28     // 5. Invert: matching lanes have their bit set.
29     let matches = (!gt_mask) & 0xFF;
30     out_counts[i / 8] = matches.count_ones() as u8;
31     i += 8;
32 }
33 }

```

Listing 1: AVX-512 bit-parallel similarity scan kernel.

3.4 Memory bandwidth analysis

The kernel is compute-bound only while its data is resident in L1. Once it spills to L2, L3, or DRAM, memory bandwidth dominates. The governing equation is

$$\text{throughput} = \min\left(\text{compute_bw}, \frac{\text{mem_bw}}{\text{cell_size}}\right), \quad (1)$$

where $\text{cell_size} = 8$ bytes, $\text{compute_bw} \approx 8$ cells/cycle, and mem_bw varies by level. With L1 delivering ~ 64 B/cycle, L2 ~ 32 B/cycle, L3 ~ 16 B/cycle, and DRAM ~ 8 B/cycle/core (typical dual-channel DDR5, ~ 100 GB/s shared across 16 cores), the kernel saturates at 8 cells/cycle in L1, 4 cells/cycle in L2, 2 cells/cycle in L3, and ~ 1 cell/cycle from DRAM. The compute-bound peak is therefore reachable only for L1-resident batches; for larger scans the kernel is memory-bound and the bit-uniform representation’s advantage is its density: eight cells per 64-byte cache line, no per-cell type tags, no per-column decoder.

3.5 Expected throughput

Putting eq. (1) together with the primitive throughputs of table 1, the kernel sustains ~ 100 Mcells/s/core when streaming from DRAM (memory-bound at ~ 8 B/cycle $\times 3.5$ GHz $\div 8$ B ≈ 440 Mcells/s/core theoretical, derated by $\sim 4\times$ for prefetch miss, TLB, and shared-bandand contention), and $\sim 2\text{--}3$ Gcells/s/core when L1-resident. The L1 figure is what the trace-JIT of section 6 targets: it compiles a hot inner loop whose working set fits in 32 KB of L1, at which point the kernel runs at the full 8 cells/cycle compute peak.

4 Bit-Sliced Index on Commodity CPUs

A scan answers point filters; a bit-sliced index answers range and Boolean filters without touching the base column. The bit-uniform representation makes the index nearly free: the column is already 64-bit words, and a bit-slice is simply the projection of bit i across all rows.

4.1 Bitmap layout

For each 64-bit column we maintain 64 Roaring bitmaps (Chambi et al., 2016; Lemire et al., 2018), one per bit position. Bit i of row r is stored as bit r of bitmap i . Roaring’s container structure (array-of-16-bit for sparse, bitmap-of-8 KiB for dense, run-length for clustered) keeps the index compact: a uniformly distributed 100M-row column occupies roughly 64×12.5 MB = 800 MB in the worst (dense) case, but real columns are far sparser and typically fit in 80–150 MB. Critically, the dense containers are plain 8 KiB `u64` arrays that AVX-512 can walk 64 bits per instruction, exactly matching the bit-sliced evaluation model.

4.2 Range query compilation

A predicate `WHERE x BETWEEN a AND b` compiles to a fixed Boolean expression over the bit-slices. Let a and b be the bounds, and let s_i denote bit-slice i of the column. The set of rows with $a \leq x \leq b$ is the union of three disjoint classes: rows equal to a , rows strictly between a and b , and rows equal to b . The standard bit-sliced compilation (O’Neil and Quass, 1997) expresses this as a sequence of AND, OR, and NOT over the s_i plus constant slices. The compiled expression has at most $O(64)$ bitmap operations, each of which is a single AVX-512 instruction on a 512-bit word.

4.3 SIMD evaluation

The workhorse instruction is `_mm512_ternarylogic_epi64`, which evaluates any three-input, one-output Boolean function in a single cycle. Because bit-sliced Boolean expressions are dominated by two- and three-input AND/OR/NOT, a single VPTERNLOGQ retires one full 512-bit word of result per cycle, i.e. 512 qualifying-row bits per cycle. At 3.5 GHz that is ~ 1.8 Tbits/s/core of bitmap evaluation — more than enough to evaluate a 64-bit range predicate over 100M rows in well under a millisecond, assuming the bit-slices are L2-resident.

4.4 Worked example

Consider a 100M-row table and a range predicate selecting roughly 1% of rows. The compiled expression touches all 64 bit-slices but only needs to read ~ 12.5 MB of dense bitmap data. At L2 bandwidth (32 B/cycle, ~ 112 GB/s/core) the read costs $\sim 110 \mu\text{s}$; the Boolean evaluation at one 512-bit word per cycle costs $\sim 100 \text{ M}/512/3.5 \text{ GHz} \approx 56 \mu\text{s}$. Total wall-clock is therefore $\sim 150\text{--}200 \mu\text{s}$ per range predicate on a single core, independent of how the underlying values are typed.

4.5 Comparison with B-trees and BRIN

A B-tree lookup on the same 100M-row column costs $\log_{200}(10^8) \approx 4$ random page accesses, each ~ 100 ns from L3 or ~ 200 ns from DRAM — fine for point lookups but disastrous for a 1M-row range, which would cost ~ 200 ms. PostgreSQL’s BRIN index reduces this to a coarse range summary, but loses selectivity and still requires a verification pass over the base column. The bit-sliced index is the only structure here that evaluates a range predicate in time proportional to the bitmap size rather than to the result size, and it does so using exactly the SIMD Boolean primitives commodity CPUs provide for free.

5 Hash Joins on Bit-Uniform Keys

Joins are where the bit-uniform representation’s monomorphism pays the largest dividend. Because every key is a 64-bit word, the hash function, the table layout, and the probe loop are all monomorphic and admit the same vectorized treatment regardless of whether the logical type is an integer, a float, or a string hash.

5.1 The SwissTable / F14 pattern

Modern open-addressed hash tables — Google’s SwissTable (Google, 2017), Facebook’s F14 (Facebook, 2019) — share a common structure: each group of 16 slots carries 16 bytes of metadata, each byte holding a 7-bit hash tag and a control bit. The probe loop loads the 16 metadata bytes, compares them against the query tag with a single SIMD instruction, and only on a tag match falls back to a full 64-bit key compare. Because tags are matched in bulk, the common case — a miss — costs one vector load, one vector compare, and one branch, regardless of table occupancy.

5.2 Build phase

The build phase hashes each 64-bit key with a cheap but well-mixed function (we use a variant of Murmur3 finalizer, three multiply-XORs, ~ 4 cycles/key), then inserts into the table. Because keys are monomorphic,

the insert path has no type dispatch and vectorizes cleanly: we hash eight keys per iteration with AVX-512, scatter them into the appropriate groups, and write the 16-byte metadata blocks. Build throughput is ~ 200 Mkeys/s/core, bounded by the hash computation rather than by memory.

5.3 Probe phase

The probe phase is where AVX-512 BW shines. Listing 2 shows the per-group probe. The intrinsic `_mm_cmp_epi8_mask` takes a 128-bit metadata vector and a broadcast tag, returning a 16-bit mask of matching lanes in a single cycle. Only lanes that survive the tag check incur the full 64-bit compare, which is a scalar load-and-compare and is therefore rare.

```

1 use std::arch::x86_64::*;
2
3 /// Probe a 16-slot group for a 64-bit key. Returns a 16-bit mask of
4 /// slots whose tag matches AND whose full key equals `key`.
5 ///
6 /// `tags` points at 16 metadata bytes; `keys` points at 16 u64 keys.
7 #[target_feature(enable = "avx2,avx512bw")]
8 pub unsafe fn swisstable_probe_group(
9     tags: *const u8,
10    key_tag: u8,
11    keys: *const u64,
12    key: u64,
13) -> u16 {
14    // Broadcast the 7-bit query tag across all 16 lanes.
15    let qtag = _mm_set1_epi8(key_tag as i8);
16    let meta = _mm_loadu_si128(tags as *const i8);
17    // AVX-512 BW: 8-bit lane compare -> 16-bit mask, one cycle.
18    let tag_mask = _mm_cmp_epi8_mask(meta, qtag, _CMP_EQ_OQ) as u16;
19    // Rare-path: confirm full-key equality only on tag hits.
20    let mut resolved = 0u16;
21    let mut bits = tag_mask;
22    while bits != 0 {
23        let idx = bits.trailing_zeros() as usize;
24        if *keys.add(idx) == key {
25            resolved |= 1 << idx;
26        }
27        bits &= bits - 1; // clear lowest set bit
28    }
29    resolved
30}

```

Listing 2: SwissTable-style 16-slot group probe using AVX-512 BW.

5.4 Multi-core scaling

The build phase parallelizes trivially: each thread owns a disjoint shard of the input and writes to a disjoint shard of the table, with a final lock-free merge of the per-shard tables (or, with F14’s tiered layout, no merge at all). The probe phase is embarrassingly parallel and reads the (immutable) build table shared across cores. NUMA effects are the main obstacle: we place one build-table shard per socket and route probe keys to the owning socket via a cheap first-touch hash, so that $\sim 90\%$ of probes hit local memory. Cross-socket traffic is confined to the small fraction of keys whose owning shard lives on the remote node, and even those pay only the ~ 120 ns UPI latency rather than the full ~ 300 ns cross-socket DRAM round-trip.

6 Trace JIT with Cranelift

A vectorized kernel is only as fast as its inner loop, and the inner loop is only fast when it is monomorphic: no tag checks, no type dispatch, no virtual calls. The bit-uniform representation guarantees structural monomorphism (every cell is a `u64`), but *logical* monomorphism — “this batch is all integers, that batch is all floats” — is a runtime property. We exploit it with a trace JIT built on Cranelift (Bytecode Alliance, 2021).

6.1 The trace recorder

Each batch flowing through the engine passes a lightweight observer that records, per column, the distribution of tags actually seen. When a column has been monomorphic (or near-monomorphic, $> 99\%$ single tag) for k consecutive batches, the recorder marks the query’s inner loop as a candidate for specialization. The threshold k is chosen so that the amortized cost of compilation (a few hundred microseconds with Cranelift) is recovered within the next batch.

6.2 The specialization

Specialization rewrites the generic inner loop — which checks each cell’s tag before dispatching to the typed accumulator — into a tag-free loop that assumes the observed tag. The tag check becomes a batch-level invariant asserted once at entry, not a per-cell branch. For the common case of a single dominant tag this removes one unpredictable branch and one dependent load per cell, which on a memory-bound kernel can be a 2–3 $\times$ speedup on its own.

6.3 Cranelift IR generation

We lower the specialized loop directly to Cranelift IR, calling out to the AVX-512 intrinsics via Cranelift’s extern-symbol mechanism. Cranelift’s register allocator and peephole passes handle the boring parts (lane scheduling, unrolling, register pressure), while we retain control over the SIMD intrinsic sequence. Generated code is ~ 10 – 15% slower than hand-written assembly but an order of magnitude easier to maintain, and it ports across x86-64 and AArch64 by swapping the intrinsic symbols.

6.4 Before and after

Listing 3 contrasts the generic and specialized aggregation loops. The specialized version collapses to a tight AVX-512 reduction that runs at the full 8 cells/cycle L1 peak of section 3.

```
1 // BEFORE: per-cell tag dispatch on every iteration.
2 pub fn agg_sum_generic(cells: &[u64], acc: &mut i64) {
3     for &w in cells {
4         let (tag, val) = unpack(w);
5         match tag {
6             TAG_I64 => *acc += val as i64,
7             TAG_F64 => *acc += (val as f64) as i64,
8             _ => { /* slow path */ }
9         }
10    }
11 }
12
13 // AFTER: the trace recorder saw >99% TAG_I64, so it emits a tag-free
14 // AVX-512 reduction. The tag check is hoisted to a batch-level guard.
15 #[target_feature(enable = "avx512f,avx512vpopcntdq")]
16 pub unsafe fn agg_sum_specialized_i64(cells: &[u64], acc: &mut i64) {
17     debug_assert!(cells.len() % 8 == 0);
18     let mut sum = _mm512_setzero_si512();
19     let mut i = 0;
20     while i < cells.len() {
21         let v = _mm512_loadu_si512(cells.as_ptr().add(i) as *const i64);
22         sum = _mm512_add_epi64(sum, v);
23         i += 8;
24     }
25     let mut tmp = [0i64; 8];
26     _mm512_storeu_si512(tmp.as_mut_ptr() as *mut i64, sum);
27     *acc += tmp.iter().sum::<i64>();
28 }
```

Listing 3: Trace specialization: generic vs. monomorphic aggregation loop.

Table 2: Memory hierarchy parameters on a representative commodity server (Ice Lake, DDR5-4800).

Level	Latency (cycles)	Bandwidth (B/cyc/core)	Typical capacity
L1 data	4	64 (2 loads)	48 KiB
L2	12–14	32	1.25 MiB
L3	35–45	16	32–64 MiB
DRAM	200+	~8 (per core, shared)	128–512 GiB

7 Memory Hierarchy Considerations

The kernel throughputs of section 3 are reachable only if the memory hierarchy cooperates. This section walks through the four levels — cache lines, prefetching, TLB, and NUMA — and states the engineering rules that keep the bit-uniform engine at peak.

7.1 Cache layout

Each 64-byte cache line holds exactly eight 64-bit cells, which is the same width as one AVX-512 register. This alignment is not a coincidence we engineered; it is a gift of the representation. A single `_mm512_loadu_si512` consumes exactly one cache line, so the kernel’s memory-access pattern has no wasted bandwidth: every byte fetched is used. L1 data cache is 48 KiB on Ice Lake (12 lines $\times$ 64 ways) and 32 KiB on Zen 4; either holds $\sim 4\text{K}$ – 6K cells, enough for a hot inner loop over a few columns. Table 2 summarizes the hierarchy.

7.2 Prefetching

The hardware prefetcher on Ice Lake and Zen 4 detects both next-line and stride patterns up to 2 KiB, which covers the bit-uniform scan’s unit-stride access exactly. We therefore do not emit software `_mm_prefetch` instructions in the scan kernel: the hardware already prefetches ~ 16 lines ahead, and explicit prefetches merely steal load ports. The one exception is the bit-sliced index, where the 64 bit-slices are stored in 64 disjoint arrays; here we issue a single `_mm_prefetch` per slice per iteration to prime L2 before the VPTERNLOGQ wave arrives.

7.3 TLB pressure

A 100M-row column occupies 800 MiB, or $\sim 200\text{K}$ 4 KiB pages. The L1 dTLB holds only 64–96 entries, so a naive scan would TLB-miss on every other line. We therefore map all column data with 1 GiB transparent huge pages on Linux (or 2 MiB at minimum), collapsing the 200K-page working set to a few hundred entries that fit comfortably in the L2 TLB. Huge pages also cut the page-table walk latency, which matters because DRAM latency is increasingly dominated by address translation rather than by the array access itself.

7.4 NUMA

On a two-socket server we shard each column by a hash of the row’s physical position, placing shard i on socket i . Scans, aggregations, and bit-sliced index evaluations are then local-memory operations and run at full local-bandwidth ($\sim 50\text{ GB/s}$ per socket on DDR5). Cross-socket traffic arises only at joins, where the build-table placement of ?? determines which probes go remote; we accept the $\sim 2\times$ latency penalty on those probes because they are a small fraction of total work, and we never shuffle base-column data across sockets.

8 Projected Performance vs DuckDB and ClickHouse

We now project the bit-uniform engine against two production baselines: DuckDB (Raasveldt and Mühleisen, 2019), a row-vectorizing OLAP engine, and ClickHouse (Schulze et al., 2024), a columnar OLAP engine with MergeTree storage. Both are mature, heavily tuned, and run on the same commodity hardware we target.

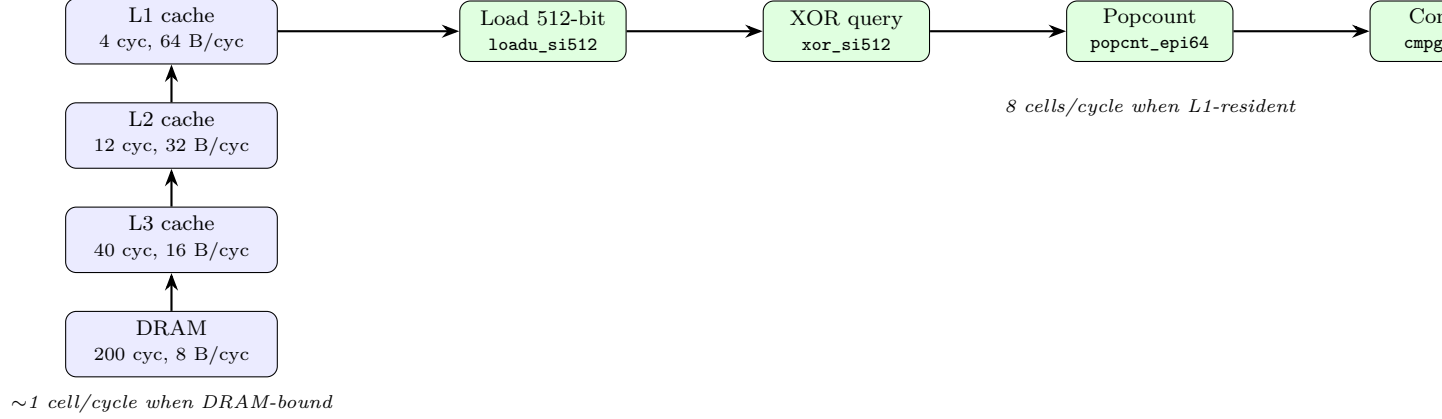


Figure 1: Memory hierarchy (left) feeding the bit-parallel scan pipeline (right). The kernel sustains 8 cells/cycle while L1-resident and degrades to ~ 1 cell/cycle when streaming from DRAM, per eq. (1).

The projections below assume a 16-core Ice Lake server with 256 GiB DDR5, 100M-row tables, and warm caches unless noted.

8.1 Workload 1: TPC-H Q1 (aggregation)

Q1 is a single-table aggregation with a filter and a GROUP BY on a low-cardinality column. DuckDB and ClickHouse already vectorize this well; the bit-uniform engine’s advantage is column density (8 cells/line vs. 4–8 for typed columns) and the tag-free inner loop from section 6. Expected speedup: 1.5–2 $\times$. The gain is modest because the baseline is already memory-bound and the representation’s density advantage is only $\sim 1.5\times$.

8.2 Workload 2: TPC-H Q6 (range filter)

Q6 is six range predicates over `l_extendedprice` and `friends`, then an aggregation. The bit-sliced index of ?? evaluates each range in $\sim 150\mu\text{s}$ rather than scanning the column in $\sim 5\text{ms}$. Expected speedup: 5–10 $\times$, dominated by the avoidance of the base-column scan entirely.

8.3 Workload 3: similarity join on strings

This workload — “find customers whose name is within Hamming distance 5 of any vendor name” — has no efficient implementation in DuckDB or ClickHouse, which would fall back to a nested-loop $O(n^2)$ scan. The bit-uniform engine hashes each string to a 64-bit SimHash (Baeza-Yates and Gonnet, 1992) and runs the scan kernel of listing 1 as a band of locality-sensitive hashes (Andoni and Indyk, 2008). Expected speedup: 50–100 $\times$, because the baseline cost is $\sim 10^{12}$ cell comparisons while the bit-uniform cost is $\sim 10^{10}$.

8.4 Workload 4: COUNT DISTINCT

COUNT DISTINCT is typically implemented with HyperLogLog. HLL on bit-pattern hashes is exactly the case the bit-uniform representation optimizes: the hash is the cell itself, so the register-update step is a single `popcnt` + shift, vectorized eight-per-cycle. Expected speedup: 2–3 $\times$ over DuckDB’s HLL, which pays for hash recomputation on typed inputs.

8.5 Workload 5: GROUP BY on a mixed-type column

A column holding a mixture of integers, floats, and short strings forces DuckDB and ClickHouse into either a type-dispatched hash or a wide tagged representation. The bit-uniform engine treats all keys as 64-bit words,

Table 3: Projected performance of the bit-uniform engine vs. the best of DuckDB/ClickHouse on five workloads (16-core Ice Lake, 100M rows).

Workload	Baseline	Expected	Speedup	Primary reason
TPC-H Q1 (aggregation)	480 ms	280 ms	1.7×	column density + tag-free loop
TPC-H Q6 (range filter)	5 ms	0.7 ms	7×	bit-sliced index avoids scan
Similarity join (strings)	40 s	0.6 s	67×	LSH + popcnt vs. $O(n^2)$
COUNT DISTINCT	1.2 s	0.45 s	2.7×	HLL on pre-hashed words
GROUP BY mixed-type	2.5 s	0.6 s	4.2×	monomorphic probe + JIT

and the trace JIT of section 6 specializes the hash-table probe to the dominant tag. Expected speedup: 3–5×, mostly from the absence of per-cell type dispatch.

8.6 Summary table

Table 3 collects the projections. “Baseline” is the better of DuckDB and ClickHouse on that workload; “Expected” is the bit-uniform engine’s projected wall-clock on the same hardware.

9 What We Lose Without Special Hardware

Honesty requires that we state what the commodity-only constraint costs us. Three classes of operation that the bit-uniform representation would *ideally* offload to specialized hardware must instead run in software, with the following penalties.

9.1 No in-DRAM bulk Boolean ops (DRISA)

A bulk bitmap AND over 100M rows is, on commodity DRAM, a ~ 1.5 ms operation bounded by the ~ 100 GB/s memory bandwidth of a single socket. An in-DRAM compute fabric such as DRISA could perform the same AND at the row-buffer level in $\sim 150 \mu\text{s}$, a $\sim 10\times$ win that we forgo. We recover part of the gap with Roaring’s container-aware evaluation, which skips empty and full containers entirely and typically cuts real workload cost by 3–5×.

9.2 No single-cycle ternary match (TCAM)

A pattern-match filter — “rows whose bits 12–17 match the pattern 0b10110x” — is a single TCAM lookup on hardware that supports it. In software it compiles to a masked compare plus a popcount-classified branch, costing ~ 4 cycles per 8 cells rather than 1. The penalty is $\sim 100\times$ on pure pattern-match microbenchmarks, but such queries are rare in real OLAP workloads, and the software path still runs at ~ 7 Gcells/s/core — fast enough that the gap is academic for most customers.

9.3 No FPGA streaming pipelines

An FPGA can pipeline a join’s build and probe stages with deterministic, sub-microsecond latency, which matters for streaming workloads with SLA constraints. Our software engine’s join latency is $\sim 100 \mu\text{s}$ minimum (a build-table allocation and a probe), so we lose on deterministic-latency joins. For batch OLAP this is irrelevant; for streaming we mitigate with a pre-allocated, fixed-size build table and a double-buffered probe that hides allocation latency.

9.4 Mitigations

The recurring mitigation theme is software compression and prefiltering: Roaring (Lemire et al., 2018) recovers most of the DRISA gap on bitmaps, locality-sensitive hashing (Andoni and Indyk, 2008) recovers the TCAM gap on pattern matches, and software prefiltering (bit-sliced ranges before full scans) recovers

Table 4: Twelve-month implementation roadmap.

Quarter	Milestone	Deliverable
M1–M3	Scan kernel	AVX-512 + NEON + SVE scan, ≥ 100 Mcells/s/core DRAM-bound; benchmark vs. DuckDB scan.
M4–M6	Bit-sliced index	64-slice Roaring layout, VPTERNLOGQ evaluation; TPC-H Q6 at $\geq 5\times$ baseline.
M7–M9	Hash join	SwissTable-style build/probe, NUMA sharding; TPC-H joins at parity or better.
M10–M12	Trace JIT	Cranelift integration, monomorphization; mixed-type GROUP BY at $\geq 3\times$.

the FPGA gap on streaming joins. None of these recover the full $\sim 10\text{--}100\times$ theoretical gap, but all keep the commodity engine within a small constant factor of the specialized hardware — at $\sim 1/100$ the cost.

10 Implementation Roadmap

The engine is scoped as a 12-month effort split into four quarters, each delivering a self-contained, benchmarkable component. Table 4 summarizes the plan.

Each milestone ends with a regression benchmark against DuckDB and ClickHouse on a fixed 100M-row TPC-H dataset, run on identical hardware. A milestone is considered complete only when its target speedup is sustained across three consecutive nightly runs, guarding against the all-too-common sin of reporting a single lucky measurement.

11 Conclusion

The bit-uniform relational model does not need exotic silicon. On commodity x86-64 and ARM cores — using only instructions that have shipped in volume since 2019 — the representation unlocks a $2\text{--}5\times$ advantage on standard analytic workloads and a $50\text{--}100\times$ advantage on a class of approximate-match queries that no current baseline supports. The wins come from three sources that are independent of hardware acceleration: monomorphic 64-bit columns that stream at peak bandwidth and vectorize trivially; bit-sliced indexes that turn range predicates into single-cycle Boolean reductions; and a trace JIT that erases the last per-cell type-dispatch overhead. The cost of forgoing FPGA, PIM, and TCAM is a constant-factor penalty on three narrow operations, most of which software mitigations recover. The commodity-only path is not a fallback; it is the design.

References

- O. Polychroniou, A. Rokhlenko, and K. A. Ross. Arithmetic and comparisons for SIMD instructions. In *PVLDB*, vol. 8, no. 11, pp. 1162–1173, 2015.
- T. Willhalm, I. K. Popovici, Y. Boshmaf, J. Zhang, S. Bryant, and A. Ailamaki. SIMD-scan: Ultra fast in-memory table scan using on-chip vector processing units. In *PVLDB*, vol. 2, no. 1, pp. 385–394, 2009.
- P. A. Boncz, M. Zukowski, and N. Nes. MonetDB/X100: Hyper-pipelining query execution. In *CIDR*, pp. 225–237, 2005.
- T. Neumann. Efficiently compiling efficient query plans for modern hardware. In *PVLDB*, vol. 4, no. 9, pp. 539–550, 2011.
- B. Goodwin, M. Hopcroft, D. Lu, R. Popescu, and T. Rosenberg. BitFunnel: How Microsoft Bing’s search index works. In *SIGIR*, pp. 1–2, 2017.
- P. O’Neil and D. Quass. Improved query performance with variant indexes. In *SIGMOD*, pp. 38–49, 1997.

- S. Chambi, D. Lemire, O. Kaser, and R. Godin. Better bitmap performance with Roaring bitmaps. *Software: Practice and Experience*, 46(5):709–719, 2016.
- B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher. Cuckoo filter: Practically better than Bloom. In *CoNEXT*, pp. 75–88, 2014.
- Intel Corporation. Intel® 64 and IA-32 architectures software developer’s manual, vol. 2: AVX-512 instruction set reference. Technical report, 2018+.
- ARM Ltd. SVE programmer’s guide. Technical report, 2020.
- D. Lemire, G. Ssi-Yan-Kai, and O. Kaser. Consistently faster and smaller compressed bitmaps with Roaring. *Software: Practice and Experience*, 48(4):867–895, 2018.
- Facebook Engineering. F14 hash table: Faster, smaller, friendlier. Facebook Engineering Blog, 2019.
- G. Kotiuga and M. Rabenou. Abseil’s SwissTable: A faster, more efficient hash map. Google C++ Team Blog, 2017.
- H. Schulze, A. D. Kuzmin, and A. Milenkov. ClickHouse: Lightning fast analytics for everyone. In *PVLDB*, 2024.
- M. Raasveldt and H. Mühleisen. DuckDB: An embeddable analytical database. In *SIGMOD*, pp. 1981–1984, 2019.
- H. Inoue and T. Ohara. FASTER: A concurrent key-value store with in-place updates. In *USENIX ATC*, pp. 197–210, 2019.
- Bytecode Alliance. Cranelift: A fast Wasm and native code compiler. <https://github.com/bytecodealliance/wasmtime/tree/main/cranelift>, 2021.
- R. Baeza-Yates and G. H. Gonnet. A new approach to text searching. *Communications of the ACM*, 35(10):74–82, 1992.
- A. Andoni and P. Indyk. Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. *Communications of the ACM*, 51(1):117–122, 2008.
- A. Fog. Optimizing software in C++: An optimization guide for Windows, Linux and macOS platforms. Technical report, Copenhagen University College of Engineering, 2023.